Phân Tích Dữ Liệu Thương Mại Điện Tử — Shopee & Lazada

> **Thành viên 3** | MapReduce · PySpark · HiveQL · Dashboard

---

0. Cài đặt thư viện & Đọc dữ liệu

```
# Cài PySpark nếu chưa có (bỏ comment dòng dưới khi dùng Colab)
# !pip install pyspark

import csv, json
from collections import defaultdict
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

ITEMS_PATH   = 'cleaned_lazada_items.csv'
REVIEWS_PATH = 'cleaned_lazada_reviews.csv'
SHOPEE_PATH  = 'cleaned_shopee.csv'

CATEGORY_MAP = {
    'beli-harddisk-eksternal': 'External HDD',
    'jual-flash-drives':       'Flash Drives',
    'beli-smart-tv':           'Smart TV',
    'shop-televisi-digital':   'Digital TV',
    'beli-laptop':             'Laptop',
}

df_items   = pd.read_csv(ITEMS_PATH)
df_reviews = pd.read_csv(REVIEWS_PATH)
df_shopee  = pd.read_csv(SHOPEE_PATH)
df_items['category_name'] = df_items['category'].map(CATEGORY_MAP)

print(f'Items   : {len(df_items):,} dòng')
print(f'Reviews : {len(df_reviews):,} dòng')
print(f'Shopee  : {len(df_shopee):,} dòng')
```

---

1. Chương trình MapReduce

> Mỗi hàm `mapper` → `shuffler` → `reducer` mô phỏng đúng pipeline Hadoop MapReduce. Có thể chạy song song trên nhiều node.

MapReduce 1 — Đếm số sản phẩm theo ngành hàng

```
def mr1_mapper(filepath):
    pairs = []
    with open(filepath, encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            cat = CATEGORY_MAP.get(row['category'], row['category'])
            pairs.append((cat, 1))
    return pairs

def mr1_shuffler(pairs):
    grouped = defaultdict(list)
    for k, v in pairs: grouped[k].append(v)
    return grouped

def mr1_reducer(grouped):
    return {k: sum(v) for k, v in grouped.items()}

# Chạy pipeline
res1 = mr1_reducer(mr1_shuffler(mr1_mapper(ITEMS_PATH)))
res1_sorted = sorted(res1.items(), key=lambda x: x[1], reverse=True)

print(f"{'Ngành hàng':<20} {'Số sản phẩm':>12}")
print('-' * 34)
for cat, cnt in res1_sorted:
    print(f'{cat:<20} {cnt:>12,}')
print(f"{'TỔNG':<20} {sum(res1.values()):>12,}")
```

MapReduce 2 — Giá trung bình theo ngành hàng

```
def mr2_mapper(filepath):
    pairs = []
    with open(filepath, encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                price = float(row['price'])
                if price > 0:
                    cat = CATEGORY_MAP.get(row['category'], row['category'])
                    pairs.append((cat, price))
            except ValueError: continue
    return pairs

def mr2_shuffler(pairs):
```

```
    grouped = defaultdict(list)
    for k, v in pairs: grouped[k].append(v)
    return grouped

def mr2_reducer(grouped):
    return {k: {'avg': sum(v)/len(v), 'min': min(v),
                'max': max(v), 'count': len(v)}
            for k, v in grouped.items()}

res2 = mr2_reducer(mr2_shuffler(mr2_mapper(ITEMS_PATH)))
res2_sorted = sorted(res2.items(), key=lambda x: x[1]['avg'], reverse=True)

print(f"{'Ngành':<18} {'Giá TB (Rp)':>16} {'Min':>14} {'Max':>16}")
print('-' * 66)
for cat, s in res2_sorted:
    print(f"{cat:<18} {s['avg']:>16,.0f} {s['min']:>14,.0f}
         {s['max']:>16,.0f}")
```

MapReduce 3 — Top sản phẩm bán chạy

```
def mr3_mapper(filepath):
    pairs = []
    with open(filepath, encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                pairs.append((row['name'].strip(), {
                    'reviews' : int(row['totalReviews']),
                    'price'   : float(row['price']),
                    'category': CATEGORY_MAP.get(row['category'],
         row['category'])
                }))
            except ValueError: continue
    return pairs

def mr3_shuffler(pairs):
    grouped = defaultdict(list)
    for k, v in pairs: grouped[k].append(v)
    return grouped

def mr3_reducer(grouped):
    return {k: max(v, key=lambda r: r['reviews']) for k, v in
         grouped.items()}

res3 = mr3_reducer(mr3_shuffler(mr3_mapper(ITEMS_PATH)))
top10 = sorted(res3.items(), key=lambda x: x[1]['reviews'], reverse=True)
         [:10]

print(f"{'#':<3} {'Tên sản phẩm':<46} {'Reviews':>9} {'Giá (Rp)':>14}")
print('-' * 76)
for i, (name, s) in enumerate(top10, 1):
```

```
    print(f"{i:<3} {(name[:43]+'...' if len(name)>43 else name):<46}"
          f" {s['reviews']:>9,} {s['price']:>14,.0f}")
```

MapReduce 4 — Top thương hiệu bán nhiều nhất

```
def mr4_mapper(filepath):
    pairs = []
    with open(filepath, encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                brand = row['brandName'].strip() or 'Unknown'
                pairs.append((brand, {'reviews': int(row['totalReviews']),
                                      'price'  : float(row['price'])}))
            except ValueError: continue
    return pairs

def mr4_shuffler(pairs):
    grouped = defaultdict(list)
    for k, v in pairs: grouped[k].append(v)
    return grouped

def mr4_reducer(grouped):
    return {k: {'total_reviews': sum(r['reviews'] for r in v),
                'product_count': len(v),
                'avg_price'    : sum(r['price'] for r in v)/len(v)}
            for k, v in grouped.items()}

res4 = mr4_reducer(mr4_shuffler(mr4_mapper(ITEMS_PATH)))
top10b = sorted(res4.items(), key=lambda x: x[1]['total_reviews'],
         reverse=True)[:10]

print(f"{'#':<3} {'Thương hiệu':<18} {'Tổng Reviews':>14} {'Số SP':>7} {'Giá
         TB':>14}")
print('-' * 60)
for i, (brand, s) in enumerate(top10b, 1):
    print(f"{i:<3} {brand:<18} {s['total_reviews']:>14,}"
          f" {s['product_count']:>7,} {s['avg_price']:>14,.0f}")
```

MapReduce 5 — Phân bố đánh giá sao

```
def mr5_mapper(filepath, rating_col='rating'):
    pairs = []
    with open(filepath, encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                r = int(float(row[rating_col]))
                if 1 <= r <= 5: pairs.append((r, 1))
            except ValueError: continue
    return pairs
```

```
def mr5_shuffler(pairs):
    grouped = defaultdict(list)
    for k, v in pairs: grouped[k].append(v)
    return grouped

def mr5_reducer(grouped):
    counts = {k: sum(v) for k, v in grouped.items()}
    total  = sum(counts.values())
    return {s: {'count': counts.get(s,0),
                'pct'  : counts.get(s,0)/total*100} for s in range(1,6)},
           total

laz_res, laz_total = mr5_reducer(mr5_shuffler(mr5_mapper(REVIEWS_PATH)))
sp_res,  sp_total  = mr5_reducer(mr5_shuffler(mr5_mapper(SHOPEE_PATH)))

print('LAZADA')
for s in range(5,0,-1):
    bar = '█'*int(laz_res[s]['pct']/3)
    print(f"  {s}★  {laz_res[s]['count']:>8,}  {laz_res[s]['pct']:>5.1f}%
          {bar}")
print('\nSHOPEE')
for s in range(5,0,-1):
    bar = '█'*int(sp_res[s]['pct']/3)
    print(f"  {s}★  {sp_res[s]['count']:>8,}  {sp_res[s]['pct']:>5.1f}%
          {bar}")
```

---

2. Phân tích bằng PySpark

> Chạy ở chế độ `local[*]` — dùng được trên máy thường. Trên cụm Hadoop/Spark thật chỉ cần đổi `.master('yarn')`.

```
# !pip install pyspark  # bỏ comment nếu chưa cài
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (SparkSession.builder
         .appName('ECommerce_Analysis')
         .master('local[*]')
         .config('spark.driver.memory','2g')
         .getOrCreate())
spark.sparkContext.setLogLevel('ERROR')

sp_items   = spark.read.csv(ITEMS_PATH,   header=True, inferSchema=True)
sp_reviews = spark.read.csv(REVIEWS_PATH, header=True, inferSchema=True)
sp_shopee  = spark.read.csv(SHOPEE_PATH,  header=True, inferSchema=True)

cat_expr = F.create_map(*[x for p in CATEGORY_MAP.items() for x in p])
sp_items = sp_items.withColumn('category_name', cat_expr[F.col('category')])
```

```
print('Spark session khởi động thành công!')
print(f'Items  : {sp_items.count():,}  |  Reviews: {sp_reviews.count():,}  |
         Shopee: {sp_shopee.count():,}')
```

Spark — Phân tích 1: Đếm sản phẩm theo ngành hàng

```
(sp_items.groupBy('category_name')
 .agg(F.count('*').alias('so_san_pham'))
 .orderBy(F.desc('so_san_pham'))
 .show(truncate=False))
```

Spark — Phân tích 2: Giá trung bình theo ngành hàng

```
(sp_items.filter(F.col('price') > 0)
 .groupBy('category_name')
 .agg(
     F.round(F.avg('price'),0).alias('gia_trung_binh_Rp'),
     F.min('price').alias('gia_min'),
     F.max('price').alias('gia_max'),
     F.count('*').alias('so_san_pham')
 )
 .orderBy(F.desc('gia_trung_binh_Rp'))
 .show(truncate=False))
```

Spark — Phân tích 3 & 4: Top sản phẩm & thương hiệu

```
# Top 10 sản phẩm
(sp_items.groupBy('name','category_name','price')
 .agg(F.max('totalReviews').alias('luot_review'))
 .orderBy(F.desc('luot_review')).limit(10).show(truncate=50))

# Top 10 thương hiệu
(sp_items.groupBy('brandName')
 .agg(F.sum('totalReviews').alias('tong_review'),
      F.count('*').alias('so_sp'),
      F.round(F.avg('price'),0).alias('gia_tb'))
 .orderBy(F.desc('tong_review')).limit(10).show(truncate=False))
```

Spark — Phân tích 5: Phân bố đánh giá sao

```
laz_total_sp = sp_reviews.count()
sp_total_sp  = sp_shopee.count()

print('Lazada:')
(sp_reviews.filter(F.col('rating').between(1,5))
 .groupBy('rating')
 .agg(F.count('*').alias('so_dg'),
      F.round(F.count('*')*100/laz_total_sp,2).alias('pct'))
 .orderBy('rating').show())

print('Shopee:')
```

```
(sp_shopee.filter(F.col('rating').between(1,5))
 .groupBy('rating')
 .agg(F.count('*').alias('so_dg'),
      F.round(F.count('*')*100/sp_total_sp,2).alias('pct'))
 .orderBy('rating').show())

spark.stop()
print('Spark stopped.')
```

---

3. Câu lệnh HiveQL

> Copy từng block SQL vào Hive CLI, Beeline, hoặc chạy:
>
> `hive -f hive_analysis.hql`

```
hive_ddl = '''
-- Tạo database & bảng
CREATE DATABASE IF NOT EXISTS ecommerce;
USE ecommerce;

CREATE TABLE lazada_items (
    itemId BIGINT, category STRING, name STRING, brandName STRING,
    url STRING, price DOUBLE, averageRating INT,
    totalReviews INT, retrievedDate STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE TBLPROPERTIES ("skip.header.line.count"="1");

LOAD DATA LOCAL INPATH '/path/to/cleaned_lazada_items.csv'
OVERWRITE INTO TABLE lazada_items;
'''
print('HiveQL DDL (tạo bảng):')
print(hive_ddl)

hive_queries = '''
-- Phân tích 1: Đếm sản phẩm
SELECT CASE category
    WHEN 'beli-harddisk-eksternal' THEN 'External HDD'
    WHEN 'jual-flash-drives'       THEN 'Flash Drives'
    WHEN 'beli-smart-tv'           THEN 'Smart TV'
    WHEN 'shop-televisi-digital'   THEN 'Digital TV'
    WHEN 'beli-laptop'             THEN 'Laptop'
    END AS category_name,
  COUNT(*) AS so_san_pham
FROM lazada_items
GROUP BY category ORDER BY so_san_pham DESC;

-- Phân tích 2: Giá trung bình
SELECT category, ROUND(AVG(price),0) AS gia_tb,
       MIN(price) AS gia_min, MAX(price) AS gia_max
FROM lazada_items WHERE price > 0
```

```
GROUP BY category ORDER BY gia_tb DESC;

-- Phân tích 3: Top sản phẩm
SELECT name, MAX(totalReviews) AS luot_review, price
FROM lazada_items GROUP BY name, price
ORDER BY luot_review DESC LIMIT 10;

-- Phân tích 4: Top thương hiệu
SELECT brandName, SUM(totalReviews) AS tong_review,
       COUNT(*) AS so_sp
FROM lazada_items GROUP BY brandName
ORDER BY tong_review DESC LIMIT 10;
'''
print('HiveQL Queries (5 phân tích):')
print(hive_queries)
```

---

4. Dashboard — 5 biểu đồ trực quan hóa

```
plt.rcParams.update({'font.size':11, 'figure.dpi':120})
COLORS = ['#3266ad','#e08c3b','#3a9e75','#d44f5a','#7b5fc5']

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Dashboard Phân Tích Thương Mại Điện Tử — Shopee & Lazada',
             fontsize=14, fontweight='bold', y=1.01)

# ── Biểu đồ 1: Bar chart sản phẩm theo ngành ──
ax1 = axes[0,0]
cats   = [c for c,_ in res1_sorted]
counts = [v for _,v in res1_sorted]
bars = ax1.bar(cats, counts, color=COLORS, edgecolor='white', linewidth=0.5)
for bar, v in zip(bars, counts):
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+50,
             f'{v:,}', ha='center', va='bottom', fontsize=9)
ax1.set_title('Bar Chart — Số SP theo ngành hàng', fontweight='bold')
ax1.set_ylabel('Số sản phẩm')
ax1.tick_params(axis='x', rotation=15)
ax1.grid(axis='y', alpha=0.3)

# ── Biểu đồ 2: Pie chart tỷ lệ ngành ──
ax2 = axes[0,1]
wedges, texts, autotexts = ax2.pie(
    counts, labels=None, autopct='%1.1f%%',
    colors=COLORS, startangle=140,
    wedgeprops={'edgecolor':'white','linewidth':1.5},
    pctdistance=0.75)
for at in autotexts: at.set_fontsize(9)
ax2.legend(wedges, cats, loc='lower center',
           bbox_to_anchor=(0.5,-0.12), ncol=2, fontsize=8)
ax2.set_title('Pie Chart — Tỷ lệ ngành hàng', fontweight='bold')
```

```
# ── Biểu đồ 3: Line chart giá ──
ax3 = axes[0,2]
sorted_cats_price = [c for c,_ in sorted(res2.items(),
                     key=lambda x:x[1]['avg'])]
avg_vals = [res2[c]['avg']/1e6 for c in sorted_cats_price]
min_vals = [res2[c]['min']/1e6 for c in sorted_cats_price]
x = range(len(sorted_cats_price))
ax3.plot(x, avg_vals, 'o-', color='#3266ad', lw=2, label='Giá TB')
ax3.plot(x, min_vals, 's--', color='#7b5fc5', lw=1.5, label='Giá Min')
ax3.fill_between(x, min_vals, avg_vals, alpha=0.1, color='#3266ad')
ax3.set_xticks(list(x))
ax3.set_xticklabels(sorted_cats_price, rotation=15, fontsize=9)
ax3.set_ylabel('Giá (triệu Rp)')
ax3.set_title('Line Chart – Xu hướng giá theo ngành', fontweight='bold')
ax3.legend(fontsize=9)
ax3.grid(alpha=0.3)

# ── Biểu đồ 4: Histogram đánh giá sao ──
ax4 = axes[1,0]
stars = [1,2,3,4,5]
laz_cnts = [laz_res[s]['count'] for s in stars]
sp_cnts  = [sp_res[s]['count']  for s in stars]
w = 0.35
xp = np.array(stars)
b1 = ax4.bar(xp-w/2, laz_cnts, w, label='Lazada', color='#3266ad',
         edgecolor='white')
b2 = ax4.bar(xp+w/2, sp_cnts,  w, label='Shopee', color='#e08c3b',
         edgecolor='white')
ax4.set_xticks(stars)
ax4.set_xticklabels([f'{s} ★' for s in stars])
ax4.set_ylabel('Số đánh giá')
ax4.set_title('Histogram – Phân bố đánh giá sao', fontweight='bold')
ax4.yaxis.set_major_formatter(plt.FuncFormatter(lambda x,_:
         f'{int(x/1000)}K'))
ax4.legend(fontsize=9)
ax4.grid(axis='y', alpha=0.3)

# ── Biểu đồ 5: Scatter plot brand ──
ax5 = axes[1,1]
brands_plot = [b for b,_ in top10b]
reviews_plot = [v['total_reviews'] for _,v in top10b]
prices_plot  = [v['avg_price'] for _,v in top10b]
sizes = [r/500 for r in reviews_plot]
sc = ax5.scatter(prices_plot, reviews_plot, s=sizes,
                 c=range(len(brands_plot)), cmap='Blues', alpha=0.8,
                 edgecolors='#3266ad', linewidths=0.8)
for i, b in enumerate(brands_plot):
    ax5.annotate(b, (prices_plot[i], reviews_plot[i]),
                 textcoords='offset points', xytext=(5,3), fontsize=8)
ax5.set_xlabel('Giá trung bình (Rp)')
ax5.set_ylabel('Tổng lượt review')
```

```
ax5.set_title('Scatter Plot — Giá TB vs Doanh số (Thương hiệu)',
         fontweight='bold')
ax5.xaxis.set_major_formatter(plt.FuncFormatter(lambda x,_: f'{x/1e6:.1f}M'))
ax5.yaxis.set_major_formatter(plt.FuncFormatter(lambda x,_:
         f'{int(x/1000)}K'))
ax5.grid(alpha=0.3)

# ── Ô trống ──
axes[1,2].axis('off')
summary = (
    'TÓM TẮT KẾT QUẢ\n\n'
    f'• Tổng sản phẩm Lazada : {len(df_items):,}\n'
    f'• Tổng đánh giá Lazada : {len(df_reviews):,}\n'
    f'• Tổng đánh giá Shopee : {len(df_shopee):,}\n\n'
    f'• Ngành nhiều nhất     : External HDD\n'
    f'• Ngành đắt nhất       : Laptop\n'
    f'• SP bán chạy nhất     : Xiaomi LED TV 32"\n'
    f'• Thương hiệu top 1    : Coocaa\n'
    f'• Rating phổ biến nhất : 5 sao (>70%)\n'
)
axes[1,2].text(0.05, 0.95, summary, transform=axes[1,2].transAxes,
              fontsize=10, va='top', family='monospace',
              bbox=dict(boxstyle='round', facecolor='#f0f4ff', alpha=0.8))

plt.tight_layout()
plt.savefig('dashboard_ecommerce.png', dpi=150, bbox_inches='tight')
plt.show()
print('Dashboard đã lưu: dashboard_ecommerce.png')
```